2026

RAPPORT DU PROJET:

DÉVELOPPEMENT D'UN VIRUS COMPAGNON EN LANGAGE C

Réalisé par: KABA Mohamed ET BALDE Amadou Dioulde
Filière: Sécurité informatique / Virologie

Encadré par: M. FRANCO Patrick ET M. BOUJU Alain
Université de la Rochelle
Année universitaire: 2025–2026

Introduction:

Dans le domaine de la cybersécurité, comprendre le fonctionnement des logiciels malveillants est indispensable pour mieux s'en protéger. Le présent rapport étudie, dans un cadre académique et contrôlé, un mécanisme de substitution de type « virus compagnon ». L'objectif n'est pas de proposer un guide pratique offensif, mais d'expliquer le principe de dissimulation, les risques de sécurité, les résultats observés en laboratoire et les contre-mesures associées. Dans ce projet, le programme étudié se présente comme un lecteur d'images nommé MediaPlayer.exe. Du point de vue de l'utilisateur, il affiche une interface graphique simple et crédible. Du point de vue de l'analyse de sécurité, il sert surtout à illustrer comment un exécutable peut détourner la confiance accordée au nom d'un programme, à son apparence et à son comportement visible.

Ce rapport est structuré de la manière suivante : * Contexte et objectifs du projet * Environnement de travail et outils mobilisés * Structure du projet et rôle des fichiers * Fonctionnement théorique du mécanisme étudié * Tests et validation avec emplacement des captures d'écran * Documents consultés et sites visités * Conclusion et recommandations de sécurité

1. Environnement de travail :

Ce projet a été réalisé sous Linux, dans un environnement de machine virtuelle universitaire, afin d'isoler les tests et de conserver un cadre pédagogique maîtrisé. Le langage C a été retenu pour son accès direct aux appels système et à la gestion des fichiers. GCC a servi à la compilation, tandis que GTK+3 a été mobilisée pour construire l'interface graphique de couverture. Certains programmes de démonstration utilisent également Gnuplot afin de rendre les exécutables cibles plus plausibles et variés. Les outils utilisés se résument ainsi : langage C pour la logique système, GCC pour la compilation, GTK+3 pour l'interface, pkg-config

pour la gestion des dépendances, et Gnuplot pour les démonstrations graphiques. Cet environnement correspond à celui présenté dans le PDF modèle fourni par l'utilisateur, qui décrit également Ubuntu, GCC, GTK+3, Gnuplot et pkg-config comme base technique du projet.

Capture 1 Environnement de développement sous Linux *Figure 1 — Environnement de développement sous Linux.* Cette capture montre la machine virtuelle, le terminal ou le dossier de travail afin de situer le contexte de l'expérimentation.

2. Structure du projet :

L'organisation du projet suit une logique simple et pédagogique. Un dossier d'images sert à alimenter l'interface graphique, un fichier principal centralise la logique de démonstration, et plusieurs programmes de test représentent les exécutables visés dans le laboratoire. Cette séparation facilite la compréhension du rôle de chaque composant et permet de documenter clairement les tests réalisés. Le fichier principal joue le rôle de cœur du prototype étudié. Les autres programmes servent de cibles de démonstration : calculatrice simple, vérification numérique, tracés graphiques et menu de choix de fonctions. Cette structure reprend l'esprit de la maquette du PDF de référence, notamment la présentation du dossier img/, du fichier MediaPlayer.c et des programmes MonPG1 à MonPG5.

Capture 2 Organisation des fichiers du projet *Figure 2 — Organisation des fichiers du projet.* Cette capture montre l'arborescence du répertoire : dossier img/, fichiers source, exécutables de test et éventuel fichier README.

3. Présentation théorique du mécanisme étudié :

Le mécanisme de type virus compagnon repose sur une logique de substitution. Au lieu de modifier directement le contenu interne d'un programme légitime, il usurpe son identité apparente afin d'être exécuté avant lui. L'utilisateur croit ouvrir un programme habituel, alors qu'un autre binaire s'exécute d'abord. Cette approche attire particulièrement l'attention en cybersécurité car elle montre que la compromission peut passer autant par l'apparence et le contexte que par la corruption directe du code. Dans le cadre du projet, l'étude reste théorique et expérimentale. Le rapport insiste sur le fait que ce type de comportement constitue une menace et doit être abordé dans une logique de prévention. Le but est donc d'analyser les mécanismes de dissimulation, la restitution apparente du service et les indices qui permettent de détecter une compromission.

4. Fonctionnement du mécanisme :

Le scénario observé en laboratoire suit généralement les étapes suivantes : recherche de fichiers exécutables dans le répertoire, vérification d'un état déjà

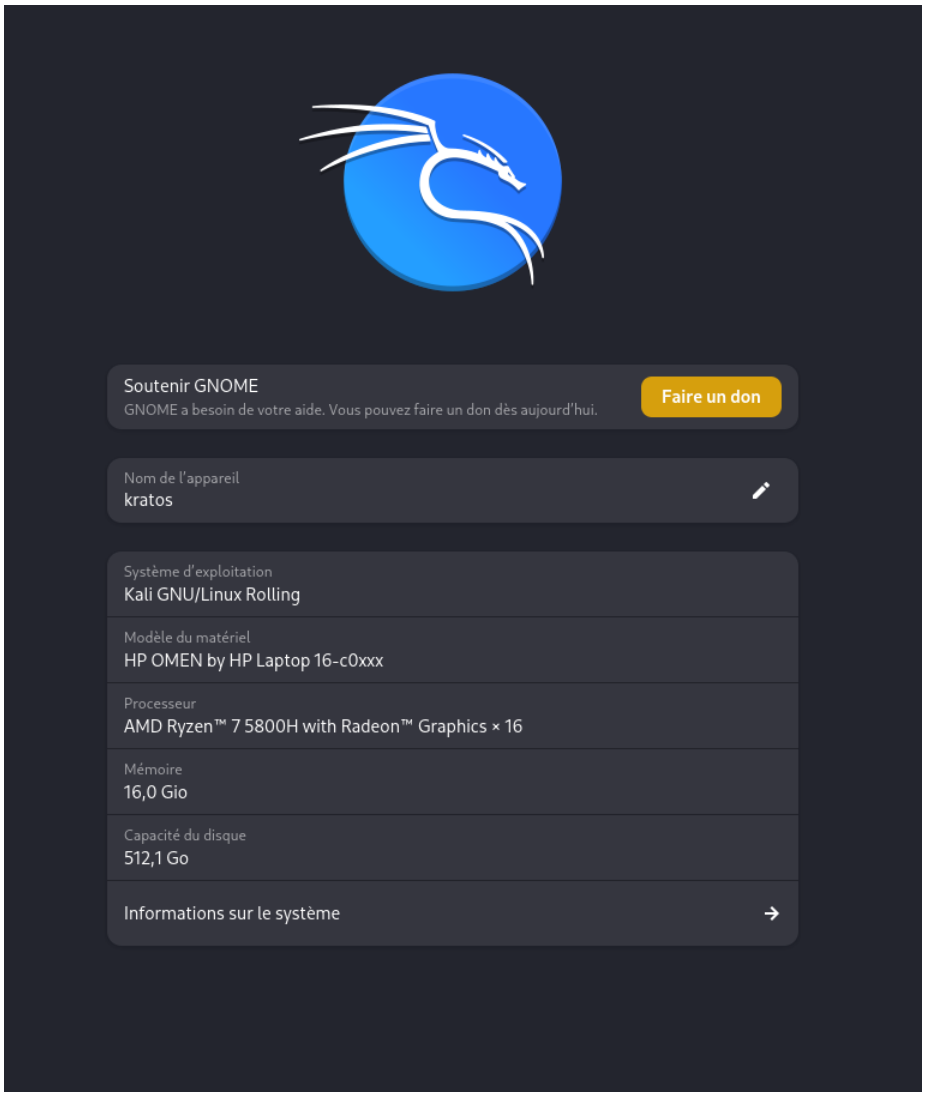


Figure 1: Environnement de développement sous Linux

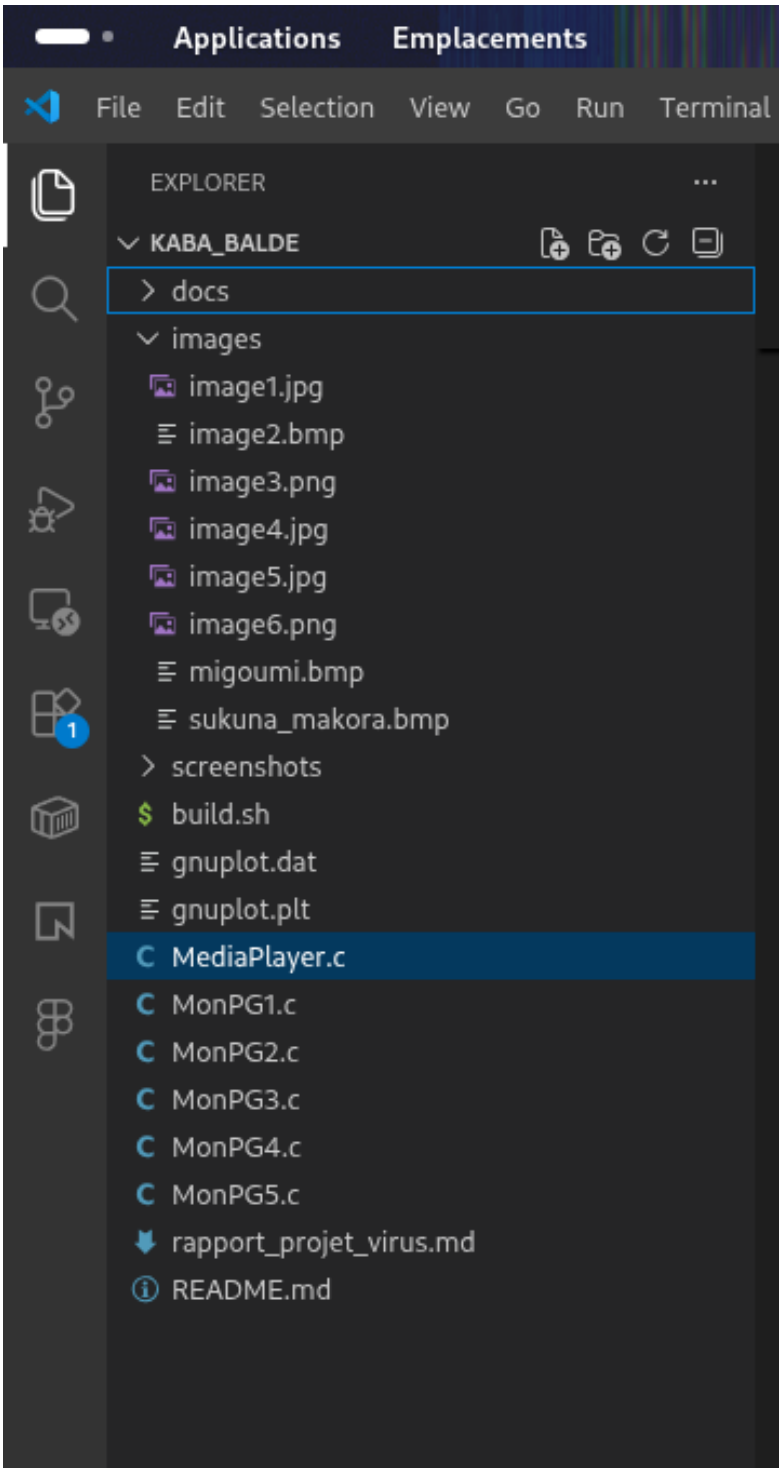


Figure 2: Organisation des fichiers du projet

traité, substitution ou renommage du programme visé, puis restitution apparente du service attendu. La logique de continuité fonctionnelle joue ici un rôle central, car un utilisateur soupçonne beaucoup moins une compromission si l'application qu'il a lancée semble finalement fonctionner normalement. Le PDF modèle présente cette logique à travers un diagramme d'état composé d'une phase de recherche, d'une vérification de sur-infection, d'une étape d'infection, d'une exécution du programme original et d'un retour vers la recherche. Cette structure peut être reprise dans le présent rapport sous une forme analytique, sans détailler de procédure offensive réutilisable.

Capture 3 Diagramme d'état du mécanisme observé *Figure 3 — Diagramme d'état du mécanisme observé.* Cette capture montre le schéma général des états : recherche, vérification, substitution, restitution du programme attendu et retour au scan.

5. Interface graphique et dissimulation visuelle :

L'interface graphique a une valeur pédagogique importante dans ce projet. Elle sert à illustrer la manière dont une façade visuelle légitime peut détourner l'attention de l'utilisateur. Avec GTK+3, il est possible d'afficher une fenêtre, une image et un bouton de navigation, donnant ainsi l'impression qu'il s'agit d'un simple visualiseur d'images. Cette stratégie correspond au rôle classique d'un cheval de Troie : l'utilisateur voit une fonction anodine et crédible pendant qu'une autre logique s'exécute en arrière-plan.

Capture 4 Fenêtre GTK affichée au lancement *Figure 4 — Fenêtre GTK affichée au lancement.* Cette capture montre la fenêtre "Mes photos" ou l'interface équivalente utilisée pour donner une apparence normale au programme.

6. Tests et validation :

La partie tests et validation permet de vérifier que le comportement observé en laboratoire est cohérent avec le scénario théorique décrit précédemment. Elle sert aussi à montrer précisément où insérer les captures d'écran demandées dans le rapport final. Chaque capture a une fonction démonstrative : prouver l'état initial, visualiser la façade graphique, constater les changements sur les fichiers, vérifier la non-répétition du traitement et observer la continuité apparente du service.

Test 1 : État initial du dossier

Avant toute expérimentation, il convient de montrer l'état vierge du répertoire de travail. Cette étape permet d'établir une base de comparaison claire pour les observations ultérieures.

Capture 5 État initial du répertoire de test *Figure 5 — État initial du répertoire de test.* Cette capture montre les fichiers présents avant l'essai, sans fichiers auxiliaires supplémentaires et sans état déjà modifié.

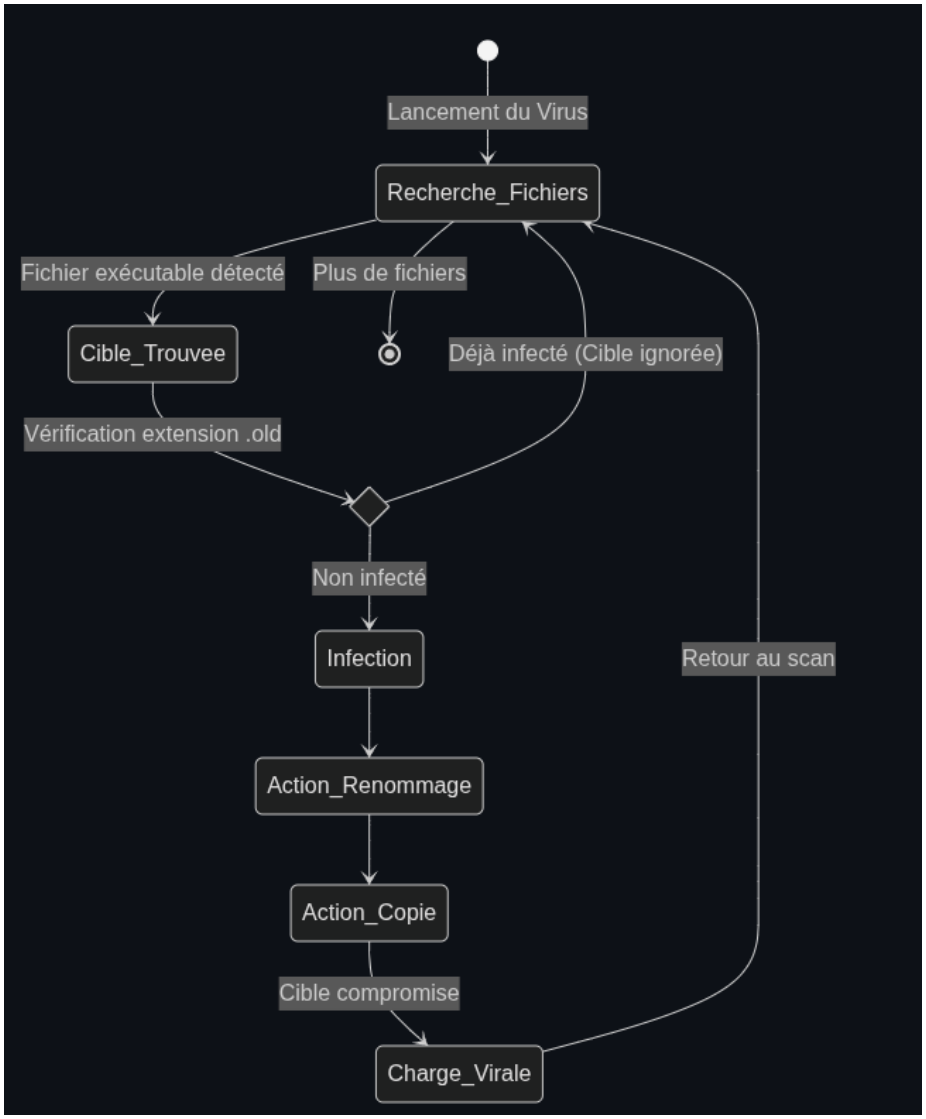


Figure 3: Diagramme d'état du mécanisme observé

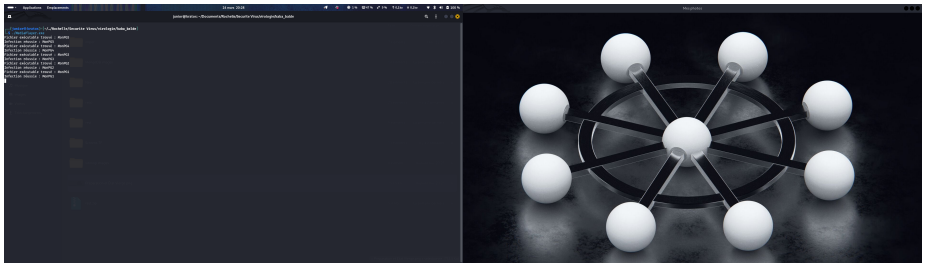


Figure 4: Fenêtre GTK affichée au lancement

Figure 5: État initial du répertoire de test

Test 2 : Lancement et interface

Le lancement du programme de couverture doit conduire à l'affichage d'une fenêtre GTK fonctionnelle. Cette étape valide la crédibilité de la façade visuelle et démontre que l'utilisateur perçoit un comportement normal au premier regard.

Capture 6 Lancement du programme et apparition de la fenêtre *Figure 6 — Lancement du programme et apparition de la fenêtre.* Cette capture montre la fenêtre ouverte après exécution du programme, avec l'image affichée et le bouton de navigation.

Test 3 : Observation du répertoire après exécution

Après le lancement, l'état du répertoire est comparé à l'état initial. Cette étape permet de constater les changements visibles dans les fichiers de test et d'expliquer leur intérêt dans l'analyse.

Capture 7 État du répertoire après exécution *Figure 7 — État du répertoire après exécution.* Cette capture met en évidence les différences observées dans le dossier de travail après l'expérimentation.

Suite des tests et validation :

Test 4 : Vérification de la non-répétition

Un point d'analyse important est de vérifier qu'un même fichier n'est pas retraité indéfiniment. Lors d'un nouveau lancement, le comportement observé doit montrer une distinction entre un état déjà pris en compte et un état nouveau. Dans le PDF modèle, cette partie correspond à la vérification dite de "sur-infection".

Capture 8 Comportement lors d'un second lancement *Figure 8 — Comportement lors d'un second lancement.* Cette capture illustre le comportement du programme lorsqu'il est relancé une seconde fois dans le même répertoire de test.

Test 5 : Continuité apparente du service

La démonstration la plus importante, du point de vue utilisateur, est la continuité apparente du service. Un programme de test relancé après la substitution doit sembler fonctionner normalement. Cette illusion explique pourquoi ce type de menace est pédagogiquement intéressant : elle exploite la confiance et l'habitude plus que la sophistication visuelle.

Capture 9 Exécution apparente d'un programme de démonstration *Figure 9 — Exécution apparente d'un programme de démonstration.* Cette capture montre qu'après l'opération observée en laboratoire, le programme de démonstration semble continuer à fournir le résultat attendu.

Figure 6: Lancement du programme et apparition de la fenêtre

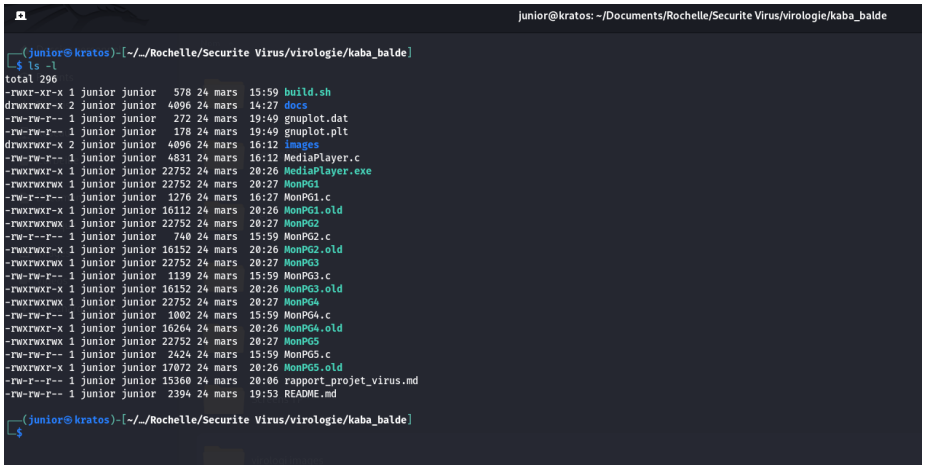


Figure 7: État du répertoire après exécution

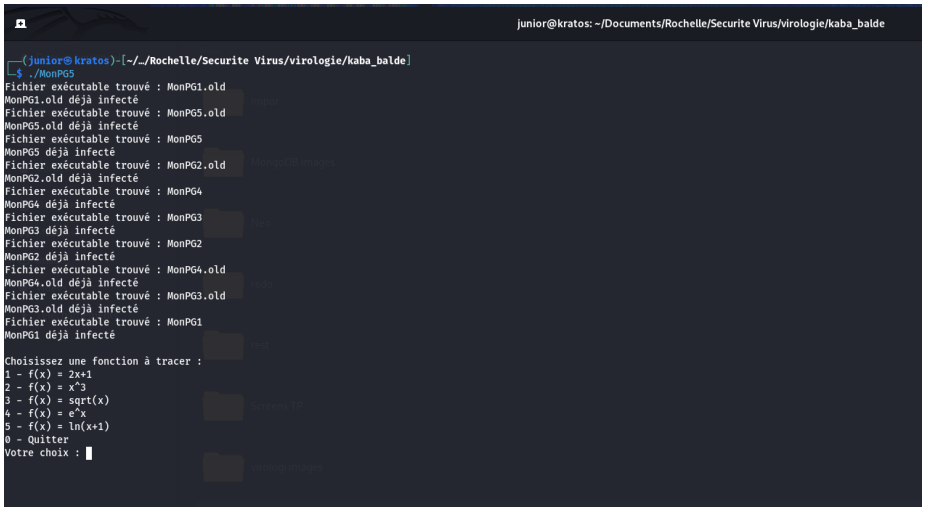


Figure 8: Comportement lors d'un second lancement

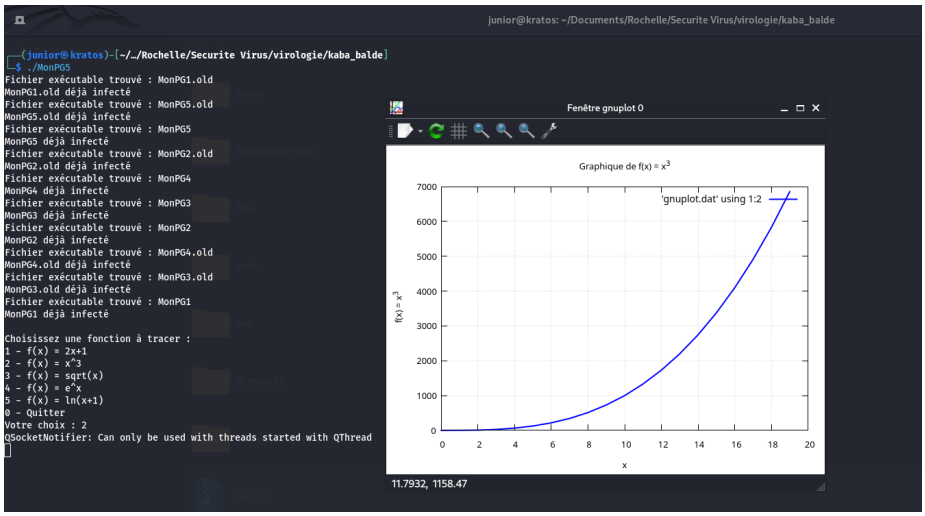


Figure 9: Exécution apparente d'un programme de démonstration

Test 6 : Programme de calcul ou vérification numérique

Il est utile d'ajouter une capture montrant un programme simple, comme une calculatrice ou un vérificateur numérique, afin de prouver que l'utilisateur continue à voir un comportement familier et cohérent.

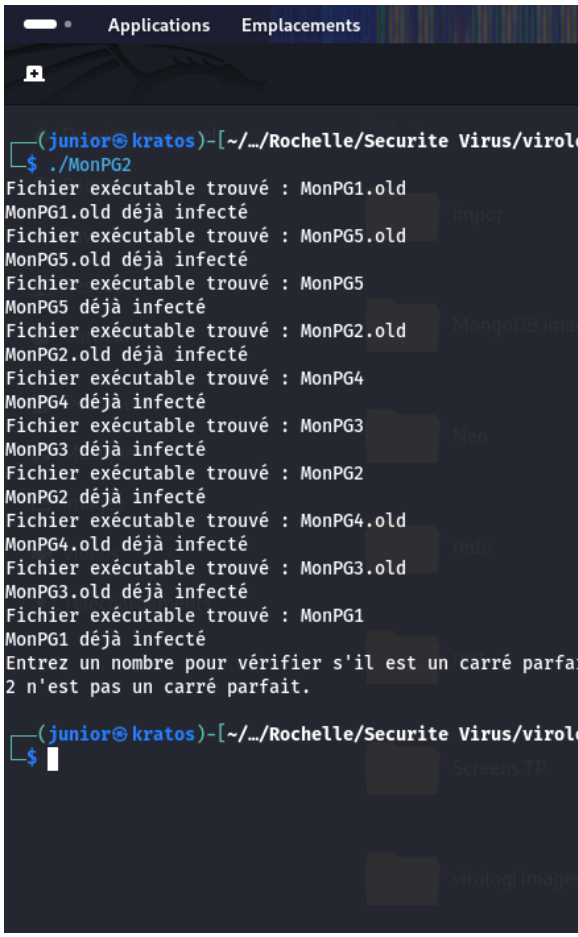
```
Applications    Emplacements

┌──(junior㉿kratos)-[~/…/Rochelle/Securite Virus/virologie/kaba_balde]
└─$ ./MonPG1
Fichier exécutable trouvé : MonPG1.old
MonPG1.old déjà infecté
Fichier exécutable trouvé : MonPG5.old
MonPG5.old déjà infecté
Fichier exécutable trouvé : MonPG5
MonPG5 déjà infecté
Fichier exécutable trouvé : MonPG2.old
MonPG2.old déjà infecté
Fichier exécutable trouvé : MonPG4
MonPG4 déjà infecté
Fichier exécutable trouvé : MonPG3
MonPG3 déjà infecté
Fichier exécutable trouvé : MonPG2
MonPG2 déjà infecté
Fichier exécutable trouvé : MonPG4.old
MonPG4.old déjà infecté
Fichier exécutable trouvé : MonPG3.old
MonPG3.old déjà infecté
Fichier exécutable trouvé : MonPG1
MonPG1 déjà infecté

------ Calculatrice simple -----
1.Addition
2.Soustraction
3.Multiplication
4.Division
5.Quitter
Votre choix
1
Entrez le premier nombre: 1589
Entrez le deuxième nombre: 156
resultat : 1745.000000

------ Calculatrice simple -----
1.Addition
2.Soustraction
3.Multiplication
4.Division
5.Quitter
Votre choix
```

Figure 10: Validation d'un programme interactif ou numérique



(Note: la capture ci-dessous est également validée par ce test :)

Capture 10 Validation d'un programme interactif ou numérique *Figure 10 — Validation d'un programme interactif ou numérique.* Cette capture montre un programme interactif en cours d'utilisation, tel qu'une calculatrice simple ou une vérification numérique.

Test 7 : Programme graphique

Enfin, l'ajout d'un programme graphique permet de montrer que le scénario n'est pas limité à des applications textuelles. Une fenêtre de tracé ou de visualisation renforce la crédibilité de l'ensemble et permet une comparaison directe avec le PDF modèle fourni.

Capture 11 Validation d'un programme graphique *Figure 11 — Validation d'un programme graphique.* Cette capture montre la fenêtre d'un tracé ou d'un graphique produit par un programme de démonstration.

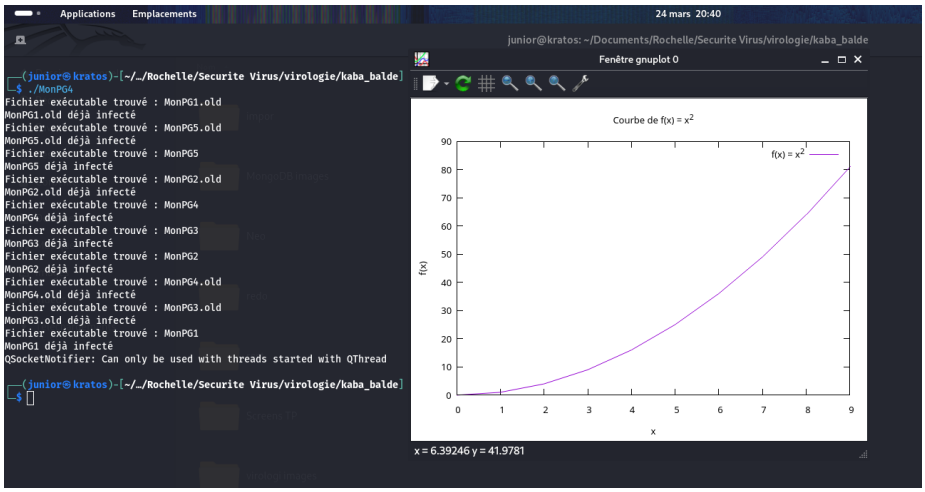


Figure 11: Validation d'un programme graphique

7. Analyse de sécurité :

L'analyse du projet montre que la force principale d'un mécanisme de substitution réside dans l'exploitation de la confiance de l'utilisateur. Lorsque le nom du programme semble légitime, que son apparence est cohérente et que le service attendu est finalement rendu, la détection immédiate devient plus difficile. Cette observation met en évidence l'importance des contrôles d'intégrité, des signatures logicielles, de la surveillance des exécutables et de la sensibilisation des utilisateurs. Le projet souligne également les limites du procédé. Une erreur de logique, une différence visible dans les métadonnées des fichiers, une modification de taille, une alerte de sécurité ou un défaut dans la restitution du programme attendu peuvent suffire à révéler l'anomalie. En environnement moderne, les solutions de sécurité comportementales, la gestion des permissions et l'inventaire applicatif réduisent fortement la portée de ce type de menace.

8. Réponses aux questions de réflexion :

Question : Pourquoi la vérification vis-à-vis du fichier cible doit-elle être double ? Réponse : Parce que tous les fichiers portant un nom donné ne sont pas forcément des exécutables légitimes. Il faut donc vérifier plusieurs éléments : type de fichier, permissions, contexte et état déjà traité.

Question : Que se produit-il si la restitution du programme attendu échoue ? Réponse : L'utilisateur remarque immédiatement une anomalie : absence de résultat, plantage, comportement inattendu ou interface incohérente.

Question : Comment l'infection pourrait-elle être amplifiée dans un scénario théorique ? Réponse : En théorie, toute amplification reposerait

sur un élargissement du périmètre exploré et sur des droits plus élevés ; du point de vue défensif, cela souligne l'importance du cloisonnement et du moindre privilège.

Question : Que se produit-il si la distinction entre programme d'origine et programme usurpateur est défaillante ? Réponse : Le comportement visible devient incohérent. Plusieurs programmes se mettraient à présenter la même façade, ce qui détruirait l'illusion et favoriserait la détection.

9. Documents consultés et sites visités :

Afin d'appuyer le cadre théorique et technique du projet, plusieurs documents de référence et sites spécialisés ont été consultés. La sélection ci-dessous privilégie des sources reconnues, adaptées à un rapport académique de sécurité informatique.

Source	Document / page consultée	Apport pour le projet
Malwarebytes Glossary	Companion virus	Définition synthétique du virus compagnon comme ancien type de virus se faisant passer pour un fichier légitime sans modifier directement le fichier original.
NIST SP 800-83 Rev.1	Guide to Malware Incident Prevention and Handling for Desktops and Laptops	Référence de sécurité pour la prévention, la détection et la gestion des incidents liés aux malwares sur les postes de travail.
GTK Documentation	GTK 3 / Getting Started	Documentation technique utilisée pour la partie interface graphique du projet.
GNU GCC Documentation	GCC manuals	Documentation consultée pour la compilation des programmes en langage C.
Linux man-pages project	Pages de manuel système	Référence pour les appels système, la gestion des fichiers, des permissions et des processus.

Webographie indicative : - https://www.malwarebytes.com/glossary/companion-virus - https://csrc.nist.gov/pubs/sp/800/83/r1/final - https://docs.gtk.org/gtk3/ - https://docs.gtk.org/gtk3/getting_started.html - https://gcc.gnu.org/onlinedocs/ - https://man7.org/linux/man-pages/

10. Documents consultés et sites visités (Complément)

Pour renforcer la qualité académique du rapport, des ressources externes fiables ont été ajoutées. Elles ont été retenues pour trois objectifs : définir correctement la notion de malware et de virus compagnon, documenter l'environnement technique Linux/GTK/GCC, et appuyer la partie défensive du mémoire par des références reconnues.

10.1 Documents web consultés

Matt Bishop, « An Overview of Computer Viruses in a Research Environment », Dartmouth College / NASA NTRS, 1991. Document théorique utilisé pour cadrer le sujet des virus informatiques dans une approche de sécurité et d'intégrité. Lien : https://ntrs.nasa.gov/api/citations/19920020387/downloads/19920020387.pdf

Murugiah Souppaya, Karen Scarfone et al., NIST SP 800-83 Rev. 1, « Guide to Malware Incident Prevention and Handling for Desktops and Laptops », NIST, 2013. Référence utilisée pour la prévention, la détection, le confinement et la reprise après incident. Lien : https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-83r1.pdf

John Aycock, Computer Viruses and Malware, Springer, 2006. Ouvrage académique mobilisé pour la culture générale sur les familles de malwares et leurs contre-mesures. Lien : https://link.springer.com/book/10.1007/0-387-34188-9

11. Conclusion :

Ce projet a permis d'analyser, dans un environnement contrôlé, les principes de fonctionnement d'un virus compagnon implémenté en langage C sous Linux. L'étude de la substitution de fichiers exécutables et de l'usage d'une interface graphique comme moyen de dissimulation a mis en évidence la complexité des menaces qui exploitent à la fois les mécanismes techniques du système et la confiance de l'utilisateur. Au-delà de l'expérimentation, ce travail souligne la nécessité de renforcer les stratégies de détection des malwares, les contrôles d'intégrité, la supervision des comportements suspects et les politiques de sécurisation des environnements d'exécution. Il met également en évidence l'importance d'une approche globale de la cybersécurité, combinant outils techniques, bonnes pratiques d'administration et sensibilisation des utilisateurs.